

# Algorithms Study Guide

A compact, professional reference for understanding how core supervised-learning algorithms work and how they move from training to live prediction.

|                                                         |                                                                  |                                                       |
|---------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------|
| 13 algorithms<br>Regression, classification & ensembles | Production-minded code<br>Training separated from live inference | Correct evaluation<br>Metrics matched to problem type |
|---------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------|

## Contents

|                                 |    |
|---------------------------------|----|
| 01 Linear Regression            | 2  |
| 02 Multiple Linear Regression   | 3  |
| 03 Polynomial Regression        | 4  |
| 04 Logistic Regression          | 5  |
| 05 K-Nearest Neighbors (KNN)    | 6  |
| 06 Decision Tree                | 7  |
| 07 Random Forest                | 8  |
| 08 Naive Bayes                  | 9  |
| 09 Support Vector Machine (SVM) | 10 |
| 10 Gradient Boosting            | 11 |
| 11 XGBoost                      | 12 |
| 12 LightGBM                     | 13 |
| 13 CatBoost                     | 14 |

How to use this guide: Read the concept and workflow first, then study the training code and live-prediction code separately. For regression, do not treat R2 as a classification-style accuracy percentage.

# 01 Linear Regression

## Supervised Learning — Regression

| WHAT IT IS                                                                                                                                                                                                    | WHERE IT FITS                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Linear Regression predicts a continuous numerical value by learning the best-fitting straight-line relationship between input features and the target. With one feature, the basic equation is $y = wx + b$ . | Use it when the target is numerical and a roughly linear relationship is reasonable. Examples: house price prediction, sales forecasting, demand estimation, and salary prediction. |

### How it works

- 01 Collect historical input-output data.
- 02 Split the data into training and testing sets.
- 03 The algorithm learns weights and an intercept that minimize prediction error.
- 04 Use the learned model to predict unseen numerical values.
- 05 Evaluate with regression metrics such as MAE, RMSE and  $R^2$ .

### Core logic

$y_{\text{hat}} = w_1x_1 + w_2x_2 + \dots + b$ . Training commonly minimizes Mean Squared Error (MSE).

### Training code

Runs during model development: learn from historical data, validate performance, then save the trained artifact.

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
df = pd.read_csv("data.csv")
X = df[["Area"]]
y = df["Price"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("MAE:", mean_absolute_error(y_test, pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("R2:", r2_score(y_test, pred))
joblib.dump(model, "linear_model.pkl")
```

### Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("linear_model.pkl")
new_house = pd.DataFrame({"Area": [1500]})
prediction = model.predict(new_house)
print("Predicted price:", prediction[0])
```

### Evaluation

Regression: MAE, MSE, RMSE and  $R^2$ . Lower MAE/RMSE is generally better;  $R^2$  closer to 1 is generally better. Always check performance on unseen validation/test data.

## 02 Multiple Linear Regression

### Supervised Learning — Regression

| WHAT IT IS                                                                                                               | WHERE IT FITS                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Multiple Linear Regression extends Linear Regression to use two or more input features to predict one continuous target. | Use it when several numerical or encoded features jointly affect a numerical target. Example: predicting house price from area, bedrooms, age and location-related features. |

### How it works

- 01 Choose multiple relevant input features.
- 02 Split data into training and testing sets.
- 03 The model learns one coefficient for each feature plus an intercept.
- 04 It combines all feature contributions to produce a prediction.
- 05 Evaluate on unseen data.

### Core logic

$y_{\text{hat}} = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$ . Each weight represents the learned contribution of a feature, holding the others fixed.

### Training code

Runs during model development: learn from historical data, validate performance, then save the trained artifact.

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
df = pd.read_csv("houses.csv")
X = df[["Area", "Bedrooms", "Age"]]
y = df["Price"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
joblib.dump(model, "multiple_linear_model.pkl")
```

### Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("multiple_linear_model.pkl")
new_house = pd.DataFrame({
    "Area": [1500],
    "Bedrooms": [3],
    "Age": [5]
})
prediction = model.predict(new_house)
print("Predicted price:", prediction[0])
```

### Evaluation

Regression: MAE, MSE, RMSE and  $R^2$ . Lower MAE/RMSE is generally better;  $R^2$  closer to 1 is generally better. Always check performance on unseen validation/test data.

## 03 Polynomial Regression

### Supervised Learning — Regression

| WHAT IT IS                                                                                                                                                                         | WHERE IT FITS                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Polynomial Regression models a curved relationship by creating polynomial features such as $x^2$ , $x^3$ and then fitting a Linear Regression model to those transformed features. | Use it when the relationship between input and target is curved rather than approximately straight. Examples: growth curves, engineering measurements, stopping distance, and some salary or demand relationships. |

### How it works

- 01 Start with the original feature(s).
- 02 Generate polynomial features using a chosen degree.
- 03 Train Linear Regression on the transformed features.
- 04 Compare degrees using validation or cross-validation.
- 05 Watch for overfitting with high degrees.

### Core logic

For degree 2:  $\hat{y} = b + w_1x + w_2x^2$ . Higher degrees add  $x^3$ ,  $x^4$ , and so on.

### Training code

Runs during model development: learn from historical data. validate performance. then save the trained artifact.

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
df = pd.read_csv("salary.csv")
X = df[["Experience"]]
y = df["Salary"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
model = LinearRegression()
model.fit(X_train_poly, y_train)
joblib.dump(poly, "poly_features.pkl")
joblib.dump(model, "poly_model.pkl")
```

### Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
poly = joblib.load("poly_features.pkl")
model = joblib.load("poly_model.pkl")
new_data = pd.DataFrame({"Experience": [7]})
new_data_poly = poly.transform(new_data)
prediction = model.predict(new_data_poly)
print("Predicted salary:", prediction[0])
```

### Evaluation

Regression: MAE, MSE, RMSE and  $R^2$ . Lower MAE/RMSE is generally better;  $R^2$  closer to 1 is generally better. Always check performance on unseen validation/test data.

# 04 Logistic Regression

## Supervised Learning — Classification

| WHAT IT IS                                                                                                                                                                                                    | WHERE IT FITS                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Logistic Regression is a classification algorithm that estimates the probability of a class. For binary classification it commonly uses the sigmoid function to map a score to a probability between 0 and 1. | Use it for binary or multiclass classification when you want a relatively simple, fast and interpretable baseline. Examples: spam detection, customer churn, disease-risk classification, and purchase prediction. |

### How it works

- 01 Prepare labeled training data.
- 02 Scale features when appropriate, especially when feature magnitudes differ.
- 03 The model learns weights using a classification loss such as log loss.
- 04 It outputs class probabilities.
- 05 A threshold or multiclass decision rule converts probabilities into predicted classes.

### Core logic

$z = w^T x + b$ ;  $\text{sigmoid}(z) = 1 / (1 + \exp(-z))$ . The sigmoid output is interpreted as the probability of the positive class.

### Training code

Runs during model development: learn from historical data. validate performance. then save the trained artifact.

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
df = pd.read_csv("customers.csv")
X = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression())
])
model.fit(X_train, y_train)
joblib.dump(model, "logistic_model.pkl")
```

### Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("logistic_model.pkl")
customer = pd.DataFrame({
    "Age": [35],
    "EstimatedSalary": [60000]
})
probability = model.predict_proba(customer)[0, 1]
prediction = model.predict(customer)[0]

print("Probability:", probability)
print("Class:", prediction)
```

### Evaluation

Classification: Accuracy, Precision, Recall, F1-score and Confusion Matrix. ROC-AUC can be useful for probabilistic binary classification. Choose the metric based on the business cost of false positives and false negatives.

# 05 K-Nearest Neighbors (KNN)

Supervised Learning — Classification / Regression

| WHAT IT IS                                                                                                                                                                                                       | WHERE IT FITS                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| KNN predicts a new sample using the labels or target values of nearby training samples. It is a distance-based, instance-based algorithm and does not learn a simple parametric equation like Linear Regression. | Use it for small-to-medium datasets where local similarity is meaningful. Examples: simple classification, recommendation by similarity, pattern recognition and some anomaly-style tasks. |

## How it works

- 01 Choose K, the number of neighbors.
- 02 Scale numerical features so distance is not dominated by large units.
- 03 Calculate the distance from the new sample to training samples.
- 04 Select the K nearest samples.
- 05 Classification uses majority voting; regression commonly averages neighbor targets.

## Core logic

A common distance is Euclidean distance:  $d(x,z) = \sqrt{\sum((x_i - z_i)^2)}$ .

## Training code

Runs during model development: learn from historical data, validate performance, then save the trained artifact.

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
df = pd.read_csv("data.csv")
X = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=5))
])
model.fit(X_train, y_train)
joblib.dump(model, "knn_model.pkl")
```

## Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("knn_model.pkl")
new_customer = pd.DataFrame({
    "Age": [30],
    "EstimatedSalary": [50000]
})
prediction = model.predict(new_customer)
print("Predicted class:", prediction[0])
```

## Evaluation

Classification: Accuracy, Precision, Recall, F1-score and Confusion Matrix. ROC-AUC can be useful for probabilistic binary classification. Choose the metric based on the business cost of false positives and false negatives.

# 06 Decision Tree

Supervised Learning — Classification / Regression

| WHAT IT IS                                                                                                                                                    | WHERE IT FITS                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A Decision Tree predicts by repeatedly splitting data using feature-based rules, creating a tree of decisions that ends in leaf nodes containing predictions. | Use it when you want a model that can capture non-linear relationships and interactions and is relatively easy to explain. Examples: loan approval, customer classification, risk scoring and regression problems. |

## How it works

- 01 Start with all training samples at the root.
- 02 Search for a useful feature and threshold/split.
- 03 Choose a split that improves a criterion such as Gini impurity or entropy for classification, or squared error for regression.
- 04 Repeat recursively until stopping rules are reached.
- 05 Predict by following the appropriate path from root to leaf.

## Core logic

For classification, Gini impurity is  $Gini = 1 - \sum(p_k^2)$ . Lower impurity means purer class groups.

## Training code

Runs during model development: learn from historical data, validate performance, then save the trained artifact.

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
df = pd.read_csv("data.csv")
X = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)
model.fit(X_train, y_train)
joblib.dump(model, "decision_tree.pkl")
```

## Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("decision_tree.pkl")
new_customer = pd.DataFrame({
    "Age": [30],
    "EstimatedSalary": [50000]
})
prediction = model.predict(new_customer)
print("Predicted class:", prediction[0])
```

## Evaluation

Classification: Accuracy, Precision, Recall, F1-score and Confusion Matrix. ROC-AUC can be useful for probabilistic binary classification. Choose the metric based on the business cost of false positives and false negatives.

# 07 Random Forest

Supervised Learning — Ensemble / Classification / Regression

| WHAT IT IS                                                                                                                                                                 | WHERE IT FITS                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Random Forest combines many Decision Trees. Each tree is trained with randomness in samples and/or features, and their predictions are combined to improve generalization. | Use it as a strong general-purpose baseline for tabular data. Examples: fraud detection, churn prediction, risk classification, feature importance analysis and regression. |

## How it works

- 01 Create many randomized training samples/features.
- 02 Train a decision tree on each randomized sample.
- 03 For classification, trees vote on the class; for regression, predictions are commonly averaged.
- 04 Combining many diverse trees usually reduces variance compared with one tree.

## Core logic

For classification, the forest prediction is the majority vote of its trees:  $y_{\text{hat}} = \text{mode}(T_1(x), T_2(x), \dots, T_n(x))$ .

## Training code

Runs during model development: learn from historical data, validate performance, then save the trained artifact.

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
df = pd.read_csv("data.csv")
X = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = RandomForestClassifier(
    n_estimators=200,
    max_depth=8,
    random_state=42
)
model.fit(X_train, y_train)
joblib.dump(model, "random_forest.pkl")
```

## Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("random_forest.pkl")
new_customer = pd.DataFrame({
    "Age": [30],
    "EstimatedSalary": [50000]
})
prediction = model.predict(new_customer)
probability = model.predict_proba(new_customer)
print("Class:", prediction[0])
print("Probabilities:", probability[0])
```

## Evaluation

Classification: Accuracy, Precision, Recall, F1-score and Confusion Matrix. ROC-AUC can be useful for probabilistic binary classification. Choose the metric based on the business cost of false positives and false negatives.

# 08 Naive Bayes

## Supervised Learning — Classification

| WHAT IT IS                                                                                                                                                        | WHERE IT FITS                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Naive Bayes is a probabilistic classifier based on Bayes' theorem. It makes a simplifying assumption that features are conditionally independent given the class. | It is especially useful for fast classification with high-dimensional features such as text. Examples: spam filtering, document classification, sentiment classification and simple recommendation/classification tasks. |

### How it works

- 01 Estimate class probabilities from training data.
- 02 Estimate feature likelihoods conditioned on each class.
- 03 Apply Bayes' theorem to compute the posterior probability of each class.
- 04 Choose the class with the highest posterior probability.

### Core logic

$P(C|X)$  is proportional to  $P(C) * P(X|C)$ . The naive assumption often factorizes  $P(X|C)$  into a product of per-feature likelihoods.

### Training code

Runs during model development: learn from historical data, validate performance, then save the trained artifact.

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
df = pd.read_csv("data.csv")
X = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = GaussianNB()
model.fit(X_train, y_train)
joblib.dump(model, "naive_bayes.pkl")
```

### Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("naive_bayes.pkl")
new_customer = pd.DataFrame({
    "Age": [30],
    "EstimatedSalary": [50000]
})
prediction = model.predict(new_customer)
print("Predicted class:", prediction[0])
```

### Evaluation

Classification: Accuracy, Precision, Recall, F1-score and Confusion Matrix. ROC-AUC can be useful for probabilistic binary classification. Choose the metric based on the business cost of false positives and false negatives.

## 09 Support Vector Machine (SVM)

Supervised Learning — Classification / Regression

| WHAT IT IS                                                                                                                                                                       | WHERE IT FITS                                                                                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SVM finds a decision boundary that separates classes while maximizing the margin between the boundary and the closest training samples. Kernels can allow non-linear boundaries. | Use it for classification problems where a clear margin is useful, especially with medium-sized datasets and well-scaled features. Examples: text classification, image classification with suitable features and biological classification. |

### How it works

- 01 Scale features because SVM is sensitive to feature magnitude.
- 02 Find a separating hyperplane with a large margin.
- 03 Allow some violations using the regularization parameter C.
- 04 Use a kernel such as RBF when a non-linear boundary is needed.
- 05 Tune C and kernel parameters using validation.

### Core logic

A linear decision boundary is  $w^T x + b = 0$ . The margin is related to  $2 / \|w\|$ , so SVM seeks a large margin while controlling classification errors.

### Training code

Runs during model development: learn from historical data, validate performance, then save the trained artifact.

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
df = pd.read_csv("data.csv")
X = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(kernel="rbf", probability=True))
])
model.fit(X_train, y_train)
joblib.dump(model, "svm_model.pkl")
```

### Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("svm_model.pkl")
new_customer = pd.DataFrame({
    "Age": [30],
    "EstimatedSalary": [50000]
})

prediction = model.predict(new_customer)
print("Predicted class:", prediction[0])
```

### Evaluation

Classification: Accuracy, Precision, Recall, F1-score and Confusion Matrix. ROC-AUC can be useful for probabilistic binary classification. Choose the metric based on the business cost of false positives and false negatives.

# 10 Gradient Boosting

Supervised Learning — Ensemble / Classification / Regression

| WHAT IT IS                                                                                                                                    | WHERE IT FITS                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gradient Boosting builds a sequence of weak decision-tree models. Each new tree focuses on reducing the errors left by the existing ensemble. | Use it for strong performance on structured/tabular data. Examples: ranking, churn prediction, fraud detection, demand prediction and risk modeling. |

## How it works

- 01 Start with a simple prediction.
- 02 Measure the errors or residuals.
- 03 Train a small tree to improve those errors.
- 04 Add the new tree to the ensemble with a learning rate.
- 05 Repeat for many iterations while controlling tree depth and learning rate.

## Core logic

Conceptually,  $F_m(x) = F_{(m-1)}(x) + \eta \cdot h_m(x)$ , where  $\eta$  is the learning rate and  $h_m$  is the next weak learner.

## Training code

Runs during model development: learn from historical data, validate performance, then save the trained artifact.

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
df = pd.read_csv("data.csv")
X = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = GradientBoostingClassifier(
    n_estimators=150,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)
model.fit(X_train, y_train)
joblib.dump(model, "gradient_boosting.pkl")
```

## Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("gradient_boosting.pkl")
new_customer = pd.DataFrame({
    "Age": [30],
    "EstimatedSalary": [50000]
})
prediction = model.predict(new_customer)
print("Predicted class:", prediction[0])
```

## Evaluation

Classification: Accuracy, Precision, Recall, F1-score and Confusion Matrix. ROC-AUC can be useful for probabilistic binary classification. Choose the metric based on the business cost of false positives and false negatives.

# 11 XGBoost

## Supervised Learning — Gradient-Boosted Trees

| WHAT IT IS                                                                                                                                       | WHERE IT FITS                                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| XGBoost is a highly optimized gradient-boosting library that adds regularization and engineering optimizations to build powerful tree ensembles. | Use it for high-performing structured/tabular ML, especially competitions and production problems where careful tuning can provide strong results. Examples: credit risk, fraud, ranking, churn and demand prediction. |

### How it works

- 01 Begin with an initial prediction.
- 02 Build trees that improve the current objective using gradient information.
- 03 Add regularization and tree constraints to control complexity.
- 04 Repeat boosting iterations.
- 05 Tune parameters using validation or cross-validation.

### Core logic

Objective =  $\sum(\text{loss}(y_i, \hat{y}_i)) + \Omega(\text{model})$ . The regularization term penalizes unnecessary model complexity.

### Training code

Runs during model development: learn from historical data. validate performance. then save the trained artifact.

```
# pip install xgboost
import pandas as pd
import joblib
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
df = pd.read_csv("data.csv")
X = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = XGBClassifier(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.05,
    eval_metric="logloss",
    random_state=42
)
model.fit(X_train, y_train)
joblib.dump(model, "xgb_model.pkl")
```

### Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("xgb_model.pkl")
new_customer = pd.DataFrame({
    "Age": [30],
    "EstimatedSalary": [50000]
})

prediction = model.predict(new_customer)
print("Predicted class:", prediction[0])
```

### Evaluation

Classification: Accuracy, Precision, Recall, F1-score and Confusion Matrix. ROC-AUC can be useful for probabilistic binary classification. Choose the metric based on the business cost of false positives and false negatives.

# 12 LightGBM

## Supervised Learning — Gradient-Boosted Trees

| WHAT IT IS                                                                                                                                                                  | WHERE IT FITS                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| LightGBM is a gradient-boosting framework designed for efficient training, especially on large tabular datasets. It uses histogram-based methods and leaf-wise tree growth. | Use it for large-scale structured/tabular datasets where training speed, memory efficiency and strong predictive performance matter. |

### How it works

- 01 Convert training data into efficient histogram-based representations.
- 02 Build boosted trees sequentially.
- 03 Use leaf-wise growth to choose promising leaves for further splitting.
- 04 Control complexity with parameters such as num\_leaves, max\_depth and learning\_rate.
- 05 Validate carefully because aggressive leaf-wise growth can overfit.

### Core logic

Like gradient boosting, it minimizes a loss through successive tree additions:  $F_m(x) = F_{(m-1)}(x) + \eta \cdot h_m(x)$ .

### Training code

Runs during model development: learn from historical data, validate performance, then save the trained artifact.

```
# pip install lightgbm
import pandas as pd
import joblib
from lightgbm import LGBMClassifier
from sklearn.model_selection import train_test_split
df = pd.read_csv("data.csv")
X = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = LGBMClassifier(
    n_estimators=200,
    learning_rate=0.05,
    num_leaves=31,
    random_state=42,
    verbosity=-1
)
model.fit(X_train, y_train)
joblib.dump(model, "lightgbm_model.pkl")
```

### Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("lightgbm_model.pkl")
new_customer = pd.DataFrame({
    "Age": [30],
    "EstimatedSalary": [50000]
})

prediction = model.predict(new_customer)
print("Predicted class:", prediction[0])
```

### Evaluation

Classification: Accuracy, Precision, Recall, F1-score and Confusion Matrix. ROC-AUC can be useful for probabilistic binary classification. Choose the metric based on the business cost of false positives and false negatives.

# 13 CatBoost

## Supervised Learning – Gradient-Boosted Trees

| WHAT IT IS                                                                                                                                                    | WHERE IT FITS                                                                                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CatBoost is a gradient-boosting framework designed to work particularly well with categorical features while reducing the need for extensive manual encoding. | Use it for tabular datasets containing categorical variables such as city, product type, occupation or customer segment. It can be useful when categorical preprocessing would otherwise be complicated. |

### How it works

- 01 Identify categorical columns.
- 02 Train ordered/target-statistics-based representations without naive leakage-prone preprocessing.
- 03 Build boosted decision trees sequentially.
- 04 Use regularization and early stopping when appropriate.
- 05 Validate on held-out data.

### Core logic

CatBoost is also gradient boosting: each iteration adds a tree that improves the selected loss while using categorical-feature handling designed to reduce leakage.

### Training code

Runs during model development: learn from historical data, validate performance, then save the trained artifact.

```
# pip install catboost
import pandas as pd
import joblib
from catboost import CatBoostClassifier
from sklearn.model_selection import train_test_split
df = pd.read_csv("customers.csv")
X = df[["Age", "City", "EstimatedSalary"]]
y = df["Purchased"]
cat_features = ["City"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = CatBoostClassifier(
    iterations=300,
    depth=6,
    learning_rate=0.05,
    verbose=False,
    random_seed=42
)
model.fit(X_train, y_train, cat_features=cat_features)
joblib.dump(model, "catboost_model.pkl")
```

### Live prediction / deployment

Runs when new data arrives: load the saved artifact, apply the same preprocessing, and call predict(). Do not retrain on every request.

```
import joblib
import pandas as pd
model = joblib.load("catboost_model.pkl")
new_customer = pd.DataFrame({

    "Age": [30],
    "City": ["Surat"],
    "EstimatedSalary": [50000]
})
prediction = model.predict(new_customer)
print("Predicted class:", prediction[0])
```

### Evaluation

Classification: Accuracy, Precision, Recall, F1-score and Confusion Matrix. ROC-AUC can be useful for probabilistic binary classification. Choose the metric based on the business cost of false positives and false negatives.